PROJECT REPORT ON

TOGETHER: A RESPONSIVE, REAL-TIME SYNCHRONIZED WATCH-PARTY WEB PLATFORM

Submitted By:

Aryan Singh
Roll No.: 2500910130053

Abhinav Pratap Singh
Roll No.: 2500910130012

Mentor:

Mr. Gurunam Singh
Assistant Professor
Dept. of Information Technology

JSS MAHAVIDYAPEETHA
DEPARTMENT OF INFORMATION TECHNOLOGY
JSS ACADEMY OF TECHNICAL EDUCATION, NOIDA

OCTOBER 2026

TABLE OF CONTENTS

Abstract **3**
1. Introduction **4**
1.1 Overview & Domain Context 4
1.2 Target Audience & Real-World Use Cases 4
2. Problem Statement and Motivation **5**
2.1 The Remote Co-Viewing Latency Challenge 5
2.2 Shortcomings of Commercial Platforms 5
2.3 Architectural Motivation for Together 6
3. Project Objectives **6**
4. System Architecture and Technology Stack **7**
4.1 Full-Stack Topology 7
4.2 Frontend Architecture (React 18 + Vite + Tailwind) 7
4.3 Real-Time Backend Engine (Node.js + Socket.IO) 8
4.4 Database Design & Supabase Schema 8
5. Detailed Implementation Modules **9**
5.1 Cryptographic Room Token Generation & Capacity Guard 9
5.2 Dual Video Player Engine (YouTube IFrame & HTML5) 9
5.3 Real-Time Clock Synchronization & Drift Compensation 10
5.4 Interactive Sidebar Chat & Floating Emoji Canvas 10
5.5 Host Authority, Voluntary Role Transfer & Failover Election 11
5.6 Tab-Isolated Guest Session Authentication 11
6. Technical Feasibility and Zero-Cost Cloud Deployment **12**
6.1 Cloud Service Matrix & Economic Feasibility 12
6.2 Production Deployment Strategy (Vercel & Render) 12
6.3 Managing Free-Tier Infrastructure Constraints 13
7. Testing, Verification & Quality Assurance **13**
7.1 Automated End-to-End Test Suite Execution 13
7.2 Security Verification & Content Guarding (DRM Handling) 14
8. Conclusion and Future Scope **14**
8.1 Summary of Contributions 14
8.2 Future Enhancements 15
9. References **15**

ABSTRACT

In modern computing and network environments, collaborative online activities—such as watching videos, webinars, and educational material synchronously—have become essential to distributed groups. However, existing commercial watch-party applications often impose intrusive account registration, aggressive subscription paywalls, third-party software installation, or fail to eliminate client-side playback drift caused by disparate network conditions.

This report documents the design, architecture, implementation, and deployment of **Together**, an open, responsive, zero-cost watch-party web application. Built with modern full-stack web standards, the system integrates a React 18, TypeScript, Vite, and Tailwind CSS frontend with a persistent Node.js, Express, and Socket.IO real-time synchronization backend. Playback synchronization is achieved through a custom clock-drift compensation algorithm with sub-1.5 second tolerance across both public YouTube streams (via the official YouTube IFrame Player API) and direct HTTPS media links (.mp4, .webm).

Security, capacity, and privacy controls are enforced using cryptographic Crockford-base32 room codes with 40+ bits of entropy, an unbreachable 5-participant capacity guard, HTML sanitization, and isolated per-tab session tokens. The complete architecture is engineered to deploy seamlessly on 100% free-tier cloud infrastructure, combining Vercel CDN hosting for the frontend, a persistent Node.js Web Service on Render, and Supabase PostgreSQL for persistence. Comprehensive end-to-end integration tests validate clock synchronization, automated host failover election, and capacity enforcement.

1. INTRODUCTION

1.1 Overview & Domain Context

The digital revolution has fundamentally altered how digital entertainment and educational video assets are consumed. Rather than engaging in isolated, solitary viewing, remote groups increasingly seek to share viewing experiences in real time. Synchronized media viewing entails running simultaneous playback across multiple heterogeneous client devices such that all participants observe identical video frames, audio tracks, pauses, and seeks simultaneously.

Together (“Watch together, wherever you are”) is a dedicated web application engineered to solve the real-world challenge of distributed co-viewing. It enables any host to instantiate an ephemeral, cryptographically secure private room with one click, share a lightweight invite link, load approved video streams, and ensure sub-second playback synchronization with up to four friends without requiring user accounts or third-party extensions.

1.2 Target Audience & Real-World Use Cases

• **Remote Friends & Families:** Enabling geographically distant groups to enjoy movies, short films, and music videos simultaneously without coordination friction.

• **Academic & Study Groups:** Allowing university students to synchronously review lecture recordings, conference presentations, and technical tutorials while maintaining an interactive chat discussion.

• **Digital Content Reviewers:** Providing a platform for creators and reviewers to inspect draft video edits collaboratively and capture live audience emoji reactions.

2. PROBLEM STATEMENT AND MOTIVATION

2.1 The Remote Co-Viewing Latency Challenge

When individuals attempt to watch remote video streams manually by coordinating via telephone or voice call, synchronization invariably degrades. Fluctuations in consumer broadband bandwidth, packet jitter, and distinct video buffer fill-rates cause playback positions to diverge by 5 to 30 seconds within minutes. Manual verbal countdowns (“3, 2, 1, Play”) are fundamentally imprecise and fail completely whenever one participant pauses or scrubs through the timeline.

2.2 Shortcomings of Commercial Platforms

Existing commercial watch-party solutions exhibit numerous operational drawbacks: (1) Mandatory Account Registration: Users are forced to provide personal identifiable information and verify emails before accessing a room. (2) Invasive Browser Extensions: Many tools mandate third-party browser add-ons that demand broad permissions to inspect web traffic, introducing severe privacy and security risks. (3) Aggressive Monetization: Freemium platforms interrupt playback with interstitial advertisements or restrict room capacity behind recurring monthly subscriptions. (4) Heavy Client Footprints: Bulky proprietary streaming clients saturate client CPU and memory resources.

2.3 Architectural Motivation for Together

Together was designed from first principles to overcome these challenges. The platform adheres strictly to four core engineering philosophies: (1) Instant Guest Access: Joining a room requires only a chosen display name; no account is ever created. (2) Deterministic State Machine: The host maintains authoritative control while participants actively correct clock drift using millisecond-accurate server sync timestamps. (3) 100% Free-Tier Viability: The architecture is built without commercial paid APIs or proprietary infrastructure. (4) Modern Aesthetic: A distraction-free cinema user interface built with responsive dark-mode styling.

3. PROJECT OBJECTIVES

The technical and functional objectives accomplished in this project include:

1. **Dual-Player Engine:** Build an adaptive media player supporting both the official YouTube IFrame Player API and native HTML5 video for direct HTTPS streams (.mp4, .webm).

2. **Real-Time WebSocket Channel:** Establish a bi-directional Socket.IO communication layer delivering millisecond-latency event propagation with automated fallback to HTTP long-polling.

3. **Capacity & Security Enforcement:** Implement an unbreachable 5-participant room capacity limit, URL-safe cryptographic base32 room tokens, and HTML input sanitization.

4. **Seamless Host Migration:** Provide voluntary host role handover and an automated failover election algorithm promoting the next oldest participant if the host disconnects.

5. **Interactive Engagement Features:** Integrate live room chat with host badges and a floating emoji reaction canvas overlay animated across the video screen.

6. **Production Cloud Deployment:** Deploy the complete system to production on Vercel and Render free tiers with automated end-to-end integration testing.

4. SYSTEM ARCHITECTURE AND TECHNOLOGY STACK

4.1 Full-Stack Topology

Together employs a decoupled, client-server distributed architecture. The React 18 Single-Page Application (SPA) runs on the client browser and establishes a persistent bi-directional WebSocket connection with the Node.js Express synchronization server. The backend maintains room states in a high-speed in-memory store with an optional PostgreSQL persistence layer via Supabase.

Layer	Technologies Utilized	Primary Responsibility
Frontend Client	React 18, Vite 6, TypeScript, Tailwind CSS, Lucide	UI rendering, dual player control, drift sync, chat & reactions
Backend Gateway	Node.js 20+, Express 4, Socket.IO 4.8, Helmet	WebSocket dispatch, host verification, capacity guard, room state
Persistence (Optional)	Supabase PostgreSQL, RLS, pg_cron	Relational room storage, audit logs, automated 24h cleanup
Cloud Hosting	Vercel Hobby (Frontend), Render (Backend)	Global low-latency edge delivery & persistent WebSockets

4.2 Database Design & Supabase Schema

The relational database schema is structured into three primary entities: (1) **rooms**: Tracks the 8-character unique room code, host session token, currently loaded video URL, playback state ('playing', 'paused', 'idle'), playback timestamp, and 24-hour expiration timestamp. (2) **participants**: Records connected session tokens, display names, join timestamps, and host flags. (3) **chat_messages**: Stores sanitized text strings, sender session tokens, and system announcement flags. All tables enforce Row-Level Security (RLS) policies.

5. DETAILED IMPLEMENTATION MODULES

5.1 Cryptographic Room Token Generation & Capacity Guard

Room codes are generated using 8 cryptographically random characters chosen from Crockford-style Base32 ('abcdefghjkmnpqrstuvwxyz23456789'), yielding 40 bits of entropy (over 1 trillion unique combinations). This prevents brute-force room discovery. When a user connects, the server enforces an atomic capacity limit: if room participants count equals 5, any additional join request is rejected immediately with an informative capacity alert.

5.2 Dual Video Player Engine

The player engine dynamically mounts either the official YouTube IFrame Player API or a native HTML5 video player based on the URL scheme: (1) **YouTube Player:** Uses window.YT.Player, configured with 100% responsive width and height. For non-host participants, controls are disabled to prevent accidental timeline scrubbing. (2) **HTML5 Player:** Uses native <video> elements with crossOrigin='anonymous'. It incorporates an Autoplay Policy Fallback modal ('Click to Join Audio & Sync') if the browser restricts unmuted autoplay before user interaction.

5.3 Real-Time Clock Synchronization & Drift Compensation

Whenever the host triggers play, pause, or seek, the server records the current playback position and the server epoch timestamp. Connected guests calculate the anticipated playback position: *Target Time = Playback Position + ((Current Time - Update Time) / 1000) * Speed*. If the absolute difference between the guest's local player time and the target time exceeds 1.5 seconds, the guest player automatically initiates a smooth seek to synchronize with the host. If the host is paused, guests remain strictly paused at the identical timestamp.

5.4 Interactive Sidebar Chat & Floating Emoji Canvas

The application includes a bi-directional live chat panel featuring auto-scrolling message feeds, timestamps, and host crown badges (■). System notifications announce participant arrivals, departures, and video changes. Below the chat feed, an emoji reaction bar allows viewers to trigger floating emoji reactions that float upward over the video screen with the sender's name tag using hardware-accelerated CSS keyframe animations.

5.5 Host Authority, Voluntary Role Transfer & Failover Election

Only the verified host session token can issue video change or playback control events. The host can transfer authority to any connected guest via a 'Pass Host' action. If the host unexpectedly disconnects, the server executes an automatic failover routine: it identifies the oldest active connection in the room, promotes them to host, and broadcasts a system notification across the room.

5.6 Tab-Isolated Guest Session Authentication

To allow developers and users to test multiple participants simultaneously on the same computer (e.g., across multiple browser tabs on localhost), the client generates a unique in-memory tab instance identifier. Session storage keys are isolated per tab: *together_token_{roomCode}_{tabId}*. This guarantees that Tab 1 (Host) and Tab 2 (Guest) maintain independent roles without token collisions.

6. TECHNICAL FEASIBILITY AND ZERO-COST CLOUD DEPLOYMENT

6.1 Cloud Service Matrix & Economic Feasibility

A foundational constraint of the project was zero-cost sustainability. The platform incurs zero recurring infrastructure expenses by operating entirely within approved cloud free tiers:

Platform	Tier	Role	Cost
Vercel	Hobby Tier	Global edge static hosting for React frontend	$0.00 / mo
Render	Free Web Service	Persistent Node.js container with WebSocket support	$0.00 / mo
Supabase	Free Tier	Managed PostgreSQL database with Row-Level Security	$0.00 / mo

6.2 Production Deployment Strategy (Vercel & Render)

Because Socket.IO requires persistent TCP connections, the backend cannot be deployed as a stateless serverless function. It is deployed as a persistent container on Render Web Services (https://together-server-3v0u.onrender.com). The frontend is deployed to Vercel CDN, configured with client-side SPA route rewrites in vercel.json. The VITE_BACKEND_URL environment variable bridges the two services, while dynamic CORS configuration permits secure cross-origin communication.

6.3 Managing Free-Tier Infrastructure Constraints

Render's free tier spins down after 15 minutes of inactivity. When a room is opened after dormancy, Render takes approximately 45 seconds to spin up. Once live, WebSocket connections remain active. Furthermore, if Supabase credentials are not supplied, the backend automatically falls back to its built-in in-memory state engine, ensuring the platform remains fully functional without external database dependencies.

7. TESTING, VERIFICATION & QUALITY ASSURANCE

7.1 Automated End-to-End Test Suite Execution

A rigorous automated integration test suite (scratch/test-e2e.js) was constructed using socket.io-client to validate room lifecycle operations under real network load. All seven critical integration checks passed successfully:

Test Case	Operational Description	Result
TC-01: Cryptographic Room Creation	Host connects and generates secure 8-character Base32 room code	**PASSED**
TC-02: Multi-User Join & Capacity Bound	4 guest sockets join sequentially; room reaches 5/5 capacity	**PASSED**
TC-03: Capacity Guard Enforcement	6th participant attempts to join; server rejects with capacity error	**PASSED**
TC-04: Video Source Propagation	Host loads YouTube video; metadata broadcasts to all participants	**PASSED**
TC-05: Playback Drift Synchronization	Host updates playback to 42.5s; all guests synchronize timestamps	**PASSED**
TC-06: Live Chat & Emoji Reaction	Guest sends chat & reaction; broadcast arrives across room	**PASSED**
TC-07: Automated Host Failover Election	Host disconnects; oldest guest is automatically elected host	**PASSED**

7.2 Security Verification & Content Guarding

Input sanitization testing verified that script injections (<script>alert(1)</script>) submitted via chat or display names are escaped to safe HTML entities. Furthermore, links pointing to DRM-protected providers (Netflix, Disney+, Prime Video) are intercepted and rejected with an informative explanation, ensuring full compliance with technical and copyright standards.

8. CONCLUSION AND FUTURE SCOPE

8.1 Summary of Contributions

The Together project demonstrates that a robust, synchronized watch-party platform can be built with production-grade engineering, deterministic playback fidelity, and zero operational costs. By leveraging React 18, Node.js, and Socket.IO, the application eliminates the barriers of mandatory registration and browser extensions while maintaining frame-accurate synchronization for up to five participants.

8.2 Future Enhancements

• **WebRTC Voice/Video Chat:** Integrating lightweight mesh WebRTC audio channels for voice conversation alongside text chat.

• **Shared Media Queues:** Allowing participants to queue multiple video tracks into a collaborative room playlist.

• **External Subtitle Synchronization:** Parsing and synchronously rendering external .vtt and .srt subtitle tracks over direct video streams.

• **Progressive Web Application (PWA):** Implementing service worker caching and web manifests for home-screen mobile installation.

9. REFERENCES

[1] Socket.IO Documentation: WebSocket & Polling Transports, Engine.IO Architecture. https://socket.io/docs/v4/

[2] YouTube Developers: YouTube Player API Reference for iframe Embeds. Google Developers. https://developers.google.com/youtube/iframe_api_reference

[3] React Documentation: Hooks, Concurrent Rendering, and Context API. https://react.dev/

[4] Vite Documentation: Next Generation Frontend Tooling & Production Bundling. https://vitejs.dev/

[5] Tailwind CSS: Utility-First CSS Framework & Animation Specifications. https://tailwindcss.com/

[6] Express.js Documentation: Routing, Middleware Architecture, and Security Controls. https://expressjs.com/

[7] Supabase Documentation: PostgreSQL Database, Row-Level Security, and Realtime Engine. https://supabase.com/docs/

[8] Together GitHub Repository: Together Watch-Party Source Code & Deployment Manifests. https://github.com/aryan06-work/watch-party